Week 07 Sequelize Practice: Answer Key

Part A — Warm-ups: Sequelize Basics & CRUD

Exercise 1: Install Sequelize & Connect to MySQL

- npm install sequelize mysql2
- "type": "module"
- import { Sequelize } from 'sequelize';
- host: 'localhost',
- dialect: 'mysql',
- export default sequelize;
- await sequelize.authenticate();

Exercise 2: Define Your First Model

- const Product = sequelize.define(
- 'Product',
- type: DataTypes.STRING,
- type: DataTypes.FLOAT,
- defaultValue: 0,
- export default Product;

Exercise 3: sync() the Table & create() Rows

- await sequelize.sync({ force: true });
- const p1 = await Product.create({
- const p2 = await Product.create({ name: 'Mouse', price: 49.99, stock: 50 });
- console.log('Created:', p1.name, p2.name);

Exercise 4: findAll() & findByPk()

- `const allProducts = await Product.findAll();`
- `const found = await Product.findByPk(1);`
- `console.log('Found:', found.name);`
- `price: { [Op.lt]: 100 },`

Exercise 5: update() & destroy()

- `product.price = 999.99;`
- `await product.save();`
- `const [affectedRows] = await Product.update(`
- `console.log('Rows updated:', affectedRows);`
- `await toDelete.destroy();`
- `await Product.destroy({ where: { stock: 0 } });`

Exercise 6: One-to-Many Relationship (hasMany / belongsTo)

- `name: { type: DataTypes.STRING, allowNull: false },`
- `Department.hasMany(Employee);`
- `Employee.belongsTo(Department);`
- `await engineering.createEmployee({ name: 'Alice', jobTitle: 'Backend Dev' });`
- `include: [ Employee ],`

Exercise 7: One-to-One Relationship (hasOne / belongsTo)

- `User.hasOne(Profile);`
- `Profile.belongsTo(User);`
- `await user.createProfile({ bio: 'Backend developer', avatar: 'sokha.png' });`
- `include: [ Profile ],`

Exercise 8: Many-to-Many Relationship (belongsToMany)

- `Student.belongsToMany(Course, { through: Enrolment });`
- `Course.belongsToMany(Student, { through: Enrolment });`
- `await alice.addCourse(math, { through: { grade: 'A' } });`
- `model: Course,`
- `attributes: ['grade']`

Part B — Core: Sequelize Relationships

Exercise 9: Fix Broken Codes

Question	Problem	Fixed Code
Q1 - Code 1	Missing the reciprocal `belongsTo` association on the Profile model, so the foreign key isn't created on Profile.	`User.hasOne(Profile);` `Profile.belongsTo(User);`
Q2 - Code 2	The relationship is defined backwards. A book belongs to an author, it does not have many authors.	`Author.hasMany(Book);` `Book.belongsTo(Author);`
Q3 - Code 3	A `hasOne` relationship generates a `set...` method, not an `add...` method.	`await user.setProfile(profile);`
Q4 - Code 4	Reciprocal `hasOne` associations create circular foreign keys. One side must hold the key using `belongsTo`.	`Manager.hasMany(Employee);` `Employee.belongsTo(Manager);`

Exercise 10: Authors & Books

- `Author.hasMany(Book);`
- `Book.belongsTo(Author);`

- `await ronan.createBook({ title: 'The Art of Code', ... });`
- `const books = await author.getBooks();`
- `await hokTim.createBook({ title: 'Young Genius', ... });`
- `include: [Book],`

Exercise 11: Attendance Tracker

- `Student.hasMany(AttendanceRecord);`
- `AttendanceRecord.belongsTo(Student);`
- `Class.hasMany(AttendanceRecord);`
- `AttendanceRecord.belongsTo(Class);`
- `Student.belongsToMany(Class, { through: AttendanceRecord });`
- `Class.belongsToMany(Student, { through: AttendanceRecord });`
- `await AttendanceRecord.create({`
- `where: { StudentId: 1, date: '2025-06-17' },`
- `include: [ Student ],`
- `const presentCount = summary.filter(r => r.status === 'present').length;`
- `const { studentId, date } = req.query;`
- `ClassId: classId,`
- `res.status(201).json(record);`